TEAM REVIEW EDITION

WS-REV-001: Workstream Review and Revision Lifecycle Specification

A locked implementation-handoff specification prepared for structured team review.

DOCUMENT	WS-REV-001
STATUS	Locked for implementation handoff
MATURITY	Design complete; not yet runtime-proven
VERSION / DATE	0.1 / 2026-07-10
OWNER	Flow Research / Workstream Engineering
SCOPE	Reviewer authority, routing, preferred-reviewer backlog, open FIFO queue, claim leases, immutable reviews, revision replay, and terminal reject behaviour

Review instruction: validate domain decisions, invariants, transaction boundaries, failure behaviour, and conformance coverage. Proposed changes should identify the exact section and affected invariant.

Contents

Use the PDF outline or the section index below to navigate the specification.

1. Purpose

This specification defines what Workstream does after a finalized submission passes the durable post-submission checker gate and becomes ready for human review.

It establishes:

- who may submit and review work;
- how reviewer authority is granted;
- how first submissions enter an open review pool;
- how revisions return to the previous reviewer as a preferred backlog;
- how preferred work falls back into an open FIFO pool;
- how a reviewer claims exactly one review at a time through a lease;
- how expired, released, or revoked leases return work safely to the queue;
- how submission versions, reviews, findings, and finding resolutions remain immutable;
- how `accept`, `needs_revision`, and `reject` affect the task and submitter assignment;
- which events must be auditable now so future reputation policy can use them later.

The design is informed by real high-volume contribution and review operations in which the global queue may be empty while individual reviewers still have revision backlogs routed to them.

---

2. Implementation Status and Proof Boundary

The coding agent MUST preserve the difference between proven behaviour and locked design.

Lifecycle area	Status
Guide source snapshot, sufficiency, and policy derivation	Proven
Manager approval of an exact submission policy	Proven
Effective policy and deterministic checker compilation	Proven
Task policy-context locking	Proven
Contributor claim, start, pre-submit, and submission finalization	Proven
Durable post-submission checker run	Proven
Transition to `review_pending`	Proven
Project reviewer grants	Locked by this specification
Preferred-reviewer backlog and open FIFO pool	Locked by this specification
Claim leases and capacity enforcement	Locked by this specification
Immutable review and revision chain	Locked by this specification
Reject and submitter-task blocking	Locked by this specification
Contribution record	Next specification
Payment and reputation policy	Deferred

The implementation begins at this precondition:

```
Submission finalized
  -> SubmissionVersion created
  -> task enters evaluation_pending
  -> durable checker completes
  -> routing_recommendation = allow_review
  -> task enters review_pending
  -> ReviewQueueEntry is created
```

A checker failure is not a human `Review` and MUST NOT be stored as `accept`, `needs_revision`, or `reject`. Automated evaluation remains upstream of this specification.

Version notation: Workstream and this lifecycle specification are release v0.1. References such as `SubmissionVersion(v1)` mean submission version number 1, while `/v1` is the independent API namespace. Neither denotes a Workstream v1.0 release.

3. Locked v0.1 Decisions

The following decisions are part of this contract and are not left to coding-agent interpretation.

1. Workstream has two major actor authority domains: Admin and Contributor.
2. Contributor project roles are `submitter`, `reviewer`, or `both`.
3. Admin roles such as Project Manager, Operator, Finance Authority, and Reputation Authority are internal Workstream roles.
4. Admin authority does not automatically grant reviewer authority.
5. An Admin who reviews work MUST also hold an active contributor-side reviewer grant for that project.
6. Skills and reputation inform reviewer-grant decisions but do not automatically grant reviewer authority in v0.1.
7. External reviewers use the same contributor and role-grant model after identity onboarding.
8. A contributor cannot review their own submission in v0.1.
9. A reviewer may hold exactly one active review lease globally in v0.1.
10. First submission versions enter the open FIFO review pool.
11. Revision submission versions return to the reviewer who issued the prior `needs_revision` decision for a time-limited preferred window.
12. A preferred-reviewer backlog and the open FIFO pool are distinct routing views over the same queue-entry model.
13. Preference time and review-lease time are separate timers.
14. When preference expires without a claim, the entry becomes open while preserving its original queue age.
15. When a claimed lease expires or is manually released, reviewer stickiness is cleared and the entry returns to the open pool.
16. Queue routing records are mutable; submission versions, reviews, findings, finding resolutions, and completed lease attempts are permanent.
17. `accept`, `needs_revision`, and `reject` are the only human review decisions.
18. `reject` is terminal for the task in v0.1.
19. Reject blocks the submitter from that task only; it does not revoke their project membership or project role grant.
20. Contribution, payment, and reputation work begins only after `accept` and is not implemented by this specification.

4. Actor and Authority Model

4.1 Major actor domains

```
Workstream Actor
├── Admin
│   ├── Project Manager
│   ├── Operator
│   ├── Finance Authority
│   ├── Reputation Authority
│   └── Other explicitly granted admin roles
│
└── Contributor
    ├── Submitter
    ├── Reviewer
    └── Both
```

These are authority domains, not necessarily mutually exclusive human identities. The same Identity Issuer subject may have an Admin profile and a Contributor profile only when each authority is explicitly granted.

4.2 Identity boundary

Identity Issuer establishes the external identity anchor. Workstream resolves that identity to a local actor and authorizes Workstream resources.

```
Identity Issuer token
  -> Workstream verifies issuer, subject, audience, validity, and scopes
  -> Workstream resolves local actor
  -> Workstream reads current admin/contributor grants
  -> Workstream authorizes the exact project, task, queue entry, lease, or review action
```

A valid identity token never proves that the actor is a submitter, reviewer, Project Manager, or Operator.

4.3 Admin authority relevant to this specification

Action	Minimum Workstream admin authority
Issue or revoke a contributor project role grant	Project Manager or Operator
View all project review queues and reviewer backlogs	Project Manager or Operator
Clear a preferred-reviewer route	Project Manager or Operator
Force-release an active review lease	Operator; Project Manager if project policy permits
Close a queue entry administratively	Operator
Correct routing metadata through a compensating event	Operator

Finance and Reputation Authorities receive no review-routing privilege solely from those roles.

4.4 Contributor authority

Contributor grant	May claim submission tasks	May claim review work
`submitter`	Yes	No
`reviewer`	No	Yes
`both`	Yes	Yes, except own submissions

5. Core Object Model

5.1 ProjectRoleGrant

`ProjectRoleGrant` is the source of contributor authority within one project.

```
ProjectRoleGrant:
  id: uuid
  project_id: uuid
  contributor_id: uuid
  role: enum [submitter, reviewer, both]
  status: enum [active, revoked]

  grant_method: enum [manual, automated]
  automation_policy_ref: uuid | null

  qualification_snapshot_ref: uuid

  granted_by: actor_id
  granted_at: timestamp
  revoked_by: actor_id | null
  revoked_at: timestamp | null
  revoked_reason: string | null
```

Qualification snapshot

`qualification_snapshot_ref` points to an immutable snapshot created when the grant is issued. It records what the Admin could evaluate at grant time without requiring the future reputation engine to exist.

```
ProjectRoleQualificationSnapshot:
  id: uuid
  project_id: uuid
  contributor_id: uuid
  skills_snapshot: object
  reputation_snapshot: object
  prior_project_work_refs: list[uuid]
  external_expertise_refs: list[string]
  captured_at: timestamp
  captured_by: actor_id
```

An unavailable reputation value is recorded explicitly as unavailable; it does not prevent a manual grant.

ProjectRoleGrant invariants

- A submitter claim requires an active grant with `role IN (submitter, both)` for the same project.
- A review claim requires an active grant with `role IN (reviewer, both)` for the same project.
- A review decision re-checks the same active grant at decision time.
- Grants are project-scoped.
- `grant_method = manual` is the only creation path enabled in v0.1.
- `grant_method = automated` is schema-reserved but MUST NOT be emitted by v0.1 code.
- Claim-time authority treats manual and future automated grants identically.
- Grant revocation never deletes historical reviews or lease attempts.

5.2 SubmissionVersion

`SubmissionVersion` is the immutable work version being evaluated.

```
SubmissionVersion:
  id: uuid
  task_id: uuid
  version_number: int
  submission_id: uuid
  artifact_hash: string
  prior_submission_version_id: uuid | null
  created_by: contributor_id
  created_at: timestamp
```

SubmissionVersion invariants

- Immutable from creation.
- `version_number` is strictly increasing per task and starts at 1.
- `prior_submission_version_id` is null only for version 1.
- Every later version points to the immediately preceding version for the same task.
- The creator is the submitter whose task assignment is being evaluated.
- The creator cannot claim the corresponding review queue entry.
- At most one canonical `Review` may exist for a submission version.
- A version whose queue entry closes without `review_recorded` may have no Review; therefore the database invariant is at most one, not unconditionally exactly one.

5.3 SubmissionFindingResponse

When a contributor creates a revision version, the submission MUST respond to every unresolved blocking finding from the prior review.

```
SubmissionFindingResponse:
  id: uuid
  submission_version_id: uuid
  finding_id: uuid
  response: string
  evidence_refs: list[string]
  created_at: timestamp
```

SubmissionFindingResponse invariants

- Immutable from creation.
- Required for every unresolved blocking finding when `version_number > 1`.
- `finding_id` must belong to the immediately prior review chain for the same task.
- It records the submitter's claim about how the finding was addressed; it does not mark the finding resolved.
- Only a later reviewer may record the canonical finding resolution.

5.4 ReviewQueueEntry

`ReviewQueueEntry` is mutable routing infrastructure for exactly one submission version.

```
ReviewQueueEntry:
  id: uuid
  project_id: uuid
  task_id: uuid
  submission_version_id: uuid

  queue_state: enum [pending, leased, closed]
  routing_mode: enum [preferred, open]
  routing_reason: enum [first_submission, revision_return, admin_assignment]

  first_queued_at: timestamp
  available_since: timestamp

  preferred_reviewer_id: uuid | null
  preference_expires_at: timestamp | null

  active_lease_id: uuid | null

  closed_at: timestamp | null
  closed_reason: enum [review_recorded, task_closed, admin_cancelled] | null
```

ReviewQueueEntry creation

For a first submission version:

```
queue_state: pending
routing_mode: open
routing_reason: first_submission
preferred_reviewer_id: null
preference_expires_at: null
first_queued_at: now
available_since: now
```

For a revision after `needs_revision`:

```
queue_state: pending
routing_mode: preferred
routing_reason: revision_return
preferred_reviewer_id: prior_review.reviewer_id
preference_expires_at: now + project.review_preference_window
first_queued_at: now
available_since: now
```

For direct Admin routing to an eligible external or internal reviewer:

```
queue_state: pending
routing_mode: preferred
routing_reason: admin_assignment
preferred_reviewer_id: selected_reviewer_id
preference_expires_at: now + project.review_preference_window
first_queued_at: now
available_since: now
```

Admin assignment is still time-limited so work cannot remain hidden indefinitely in one reviewer backlog.

ReviewQueueEntry invariants

- Exactly one queue entry per submission version.
- `routing_mode = preferred` requires both a preferred reviewer and a future preference expiry.
- `routing_mode = open` requires preferred-reviewer fields to be null.
- `queue_state = leased` requires a valid `active_lease_id`.
- `queue_state IN (pending, closed)` requires `active_lease_id = null`.
- `queue_state = closed` requires `closed_at` and `closed_reason`.

- Closed entries cannot return to pending.
- Queue entries are never hard-deleted while audit retention applies.

5.5 ReviewLease

Every claim creates a separate `ReviewLease`. Lease attempts are not overwritten because they are future reviewer-reputation evidence.

```
ReviewLease:
  id: uuid
  review_queue_entry_id: uuid
  reviewer_id: contributor_id
  status: enum [active, consumed, released, expired, revoked]

  claimed_at: timestamp
  expires_at: timestamp
  closed_at: timestamp | null
  close_reason: enum [review_recorded, manual_release, lease_expired, grant_revoked,
admin_override] | null
```

ReviewLease invariants

- A reviewer may have at most one active lease globally in v0.1.
- A queue entry may have at most one active lease.
- A lease may be created only while the queue entry is pending.
- A lease may be consumed only through successful canonical Review creation.
- Terminal lease statuses never change.
- Lease timestamps use server/database time, not client time.
- Expired or revoked leases cannot authorize a Review.

5.6 Review

`Review` is the immutable canonical human decision against one submission version.

```
Review:
  id: uuid
  submission_version_id: uuid
  review_queue_entry_id: uuid
  review_lease_id: uuid

  reviewer_id: contributor_id
  decision: enum [accept, needs_revision, reject]
  decision_reason: string | null

  prior_review_id: uuid | null
  reviewed_at: timestamp
  lease_duration_actual: interval
```

Review invariants

- Immutable from creation.
- Unique constraint on `submission_version_id`.
- Reviewer must own the referenced active lease.
- Lease must belong to the referenced queue entry.
- Queue entry must belong to the referenced submission version.

- Reviewer grant must still be active at decision time.
- Reviewer cannot be the submission creator.
- Version 1 review has `prior_review_id = null`.
- Review of version N points to the Review of version N-1.
- `accept` and `reject` are terminal for the submission/review chain.
- `needs_revision` permits a later submission version if the submitter continues.
- `decision_reason` is required for `needs_revision` and `reject`.

The review and submission histories are two synchronized predecessor chains, not doubly linked lists. No forward pointer is required.

5.7 ReviewFinding

Findings are immutable records attached to a Review.

```
ReviewFinding:
  id: uuid
  review_id: uuid
  description: string
  severity: enum [blocking, advisory]
  evidence_refs: list[string]
  created_at: timestamp
```

Decision and finding rules

- `accept`: findings are optional; any finding present must be advisory.
- `needs_revision`: findings are required and at least one finding must be blocking.
- `reject`: `decision_reason` is required; findings are optional but recommended.
- A finding is never edited to record its later resolution.

5.8 FindingResolution

`FindingResolution` records the later reviewer's verdict about a prior finding without mutating that finding.

```
FindingResolution:
  id: uuid
  finding_id: uuid
  submission_version_id: uuid
  resolution_status: enum [resolved, unresolved, not_applicable]
  rationale: string
  recorded_by_review_id: uuid
  recorded_at: timestamp
```

FindingResolution invariants

- Immutable from creation.
- Created only as part of reviewing a later submission version.
- Exactly one resolution per `finding_id` and reviewed `submission_version_id`.
- Every prior unresolved blocking finding must receive a resolution record in the next Review transaction.
- `accept` is invalid while any prior blocking finding is recorded as unresolved.
- A later `needs_revision` may leave prior findings unresolved and add new findings.

5.9 TaskAssignment extension

`TaskAssignment` is extended to preserve reject semantics.

```
TaskAssignment:
  id: uuid
  task_id: uuid
  contributor_id: uuid
  status: enum [active, completed, blocked]

  blocked_at: timestamp | null
  blocked_reason_review_id: uuid | null
```

TaskAssignment invariants

- `blocked` is scoped to one contributor-task relationship.
- A blocked contributor can never reclaim the same task.
- Blocking does not revoke a project role grant.
- Blocking does not affect other task assignments.
- `blocked_reason_review_id` points to the terminal reject Review.

6. Project Review Policy Fields

The project policy context locked to the task MUST include:

```
ProjectReviewPolicy:
  version: string
  review_preference_window: interval
  review_lease_duration: interval
  max_active_review_leases_per_reviewer: int  # fixed to 1 in v0.1
  self_review_allowed: bool                   # fixed to false in v0.1
  reject_policy: enum [close_task]            # only v0.1 value
```

These values are task-locked. A later project-policy change does not silently alter an already queued submission version.

7. Queue Views and Ordering

7.1 Preferred-reviewer backlog

A preferred backlog contains pending entries where:

```
routing_mode = preferred
AND preferred_reviewer_id = current_reviewer
AND preference_expires_at > now
```

This backlog explains why an individual reviewer may still have review work while the open project queue is empty.

Preferred entries are ordered by `first_queued_at ASC`.

7.2 Open FIFO pool

The open pool contains pending entries where:

```
routing_mode = open
```

Only entries for which the current actor is eligible are returned.

Open entries are ordered by:

```
first_queued_at ASC, id ASC
```

The ID tie-breaker makes ordering deterministic when timestamps match.

7.3 Reviewer next-work ordering

When Workstream selects the next claimable item for a reviewer:

1. Eligible preferred-backlog entries for that reviewer, oldest first.
2. Eligible open-pool entries, oldest first.

The server MUST NOT return another claimable entry if the reviewer already holds an active lease.

7.4 FIFO fairness on requeue

`first_queued_at` is immutable. Lease expiry and release update `available_since` but do not reset queue age.

```
first_queued_at = original queue entry time
available_since = time entry most recently became claimable
```

This prevents a contributor from losing queue priority because a reviewer claimed and stalled.

---

8. Reviewer Eligibility

A reviewer is eligible to claim a queue entry only if all conditions are true:

1. Identity token is valid for Workstream.
2. Local actor resolves to a Contributor profile.
3. Contributor has an active `ProjectRoleGrant(reviewer|both)` for the queue entry's project.
4. Contributor is not the creator of the submission version.
5. Contributor has no active review lease anywhere in Workstream.
6. Contributor is not suspended or otherwise restricted by current Workstream policy.
7. Queue entry is pending.
8. If preference is active, contributor is the preferred reviewer.
9. No canonical Review already exists for the submission version.

Eligibility is checked at claim time and again at decision time.

---

9. State Machines

9.1 ReviewQueueEntry state machine

```
pending
  -> leased          authorized atomic claim
  -> closed          task/admin closure before review

leased
  -> pending         active lease released, expired, or revoked
  -> closed          canonical Review recorded

closed
  -> terminal
```

Preference expiry does not change `queue_state`; it changes only routing:

```
pending + preferred
  -> pending + open
```

9.2 ReviewLease state machine

```
active
  -> consumed        canonical Review recorded
  -> released        reviewer voluntarily releases
  -> expired         lease duration passes
  -> revoked         reviewer grant revoked or admin force-release

consumed | released | expired | revoked
  -> terminal
```

9.3 Task review states

The queue state does not need to make the Task oscillate between pending and under-review. Queue and lease records provide that operational detail.

```
evaluation_pending
  -> review_pending      durable checker allows human review

review_pending
  -> needs_revision      Review decision
  -> accepted            Review decision
  -> closed              Review decision = reject

needs_revision
  -> in_progress         contributor continues revision work
  -> closed              contributor withdraws or admin closes under separate policy
```

When reject closes the task:

```
Task.status: closed
Task.closed_reason: review_rejected
```

10. Timer Semantics

10.1 Preference timer

The preference timer starts when a revision queue entry is created.

Purpose:

- preserve reviewer continuity;
- keep the reviewer familiar with the findings and prior evidence;

- avoid forcing a new reviewer to reconstruct the chain unnecessarily.

It does not reserve reviewer capacity and does not create a lease.

On preference expiry:

```
routing_mode -> open
preferred_reviewer_id -> null
preference_expires_at -> null
first_queued_at -> unchanged
available_since -> now
audit ReviewerPreferenceExpired
```

10.2 Lease timer

The lease timer starts only when an eligible reviewer successfully claims the queue entry.

Purpose:

- enforce one-review-at-a-time capacity;
- prevent indefinite work hoarding;
- allow automatic recovery when a reviewer stops working.

On lease expiry:

```
ReviewLease.status -> expired
ReviewLease.closed_at -> now
ReviewLease.close_reason -> lease_expired

ReviewQueueEntry.queue_state -> pending
ReviewQueueEntry.routing_mode -> open
ReviewQueueEntry.active_lease_id -> null
ReviewQueueEntry preferred fields -> null
ReviewQueueEntry.first_queued_at -> unchanged
ReviewQueueEntry.available_since -> now

audit ReviewerLeaseExpired
```

Lease expiry burns reviewer stickiness. The expired reviewer may later reclaim the entry from the open pool if still eligible, but receives no renewed preference.

11. Core Operations

11.1 Create queue entry after checker admission

Preconditions:

- task is `evaluation_pending`;
- durable checker run is completed;
- routing recommendation is `allow_review`;
- submission version has no queue entry.

Atomic effect:

```
create ReviewQueueEntry
task.status -> review_pending
record ReviewQueueEntryCreated
record TaskStatusChanged(evaluation_pending -> review_pending)
```

Idempotency is required. Reprocessing the same checker completion MUST return or preserve the existing queue entry.

11.2 Claim review

Claim is an atomic compare-and-set operation.

Preconditions:

- actor passes all reviewer eligibility checks;
- queue entry is pending;
- no active lease exists for the queue entry;
- reviewer has no active lease;
- preferred-routing rules permit this reviewer.

Atomic effect:

```
create ReviewLease(status=active)
ReviewQueueEntry.queue_state -> leased
ReviewQueueEntry.active_lease_id -> new lease ID
record ReviewerClaimedTask
```

The implementation MAY use `SELECT ... FOR UPDATE`, an atomic conditional `UPDATE`, or another database-safe compare-and-set strategy. It MUST NOT perform an unprotected read followed by a write.

11.3 Release claimed review

Only the active leased reviewer or authorized Admin may release.

Reviewer release effect:

```
lease -> released/manual_release
queue entry -> pending/open
stickiness -> cleared
available_since -> now
first_queued_at -> unchanged
record ReviewerReleasedTask
```

Admin force-release effect:

```
lease -> revoked/admin_override
queue entry -> pending/open
stickiness -> cleared
record ReviewerLeaseForceReleased
record AdminOverrideApplied(reason required)
```

11.4 Decline preferred review before claim

The preferred reviewer may decline an unclaimed preferred entry without first creating a lease.

Effect:

```
routing_mode -> open
preferred fields -> null
first_queued_at -> unchanged
available_since -> now
record ReviewerDeclinedPreference
```

This prevents work from waiting for the full preference window when the previous reviewer already knows they cannot continue.

11.5 Record review decision

Decision creation is one database transaction.

Preconditions:

- reviewer owns the active lease;
- lease is not expired according to server/database time;
- reviewer grant remains active;
- reviewer is not the submission creator;
- queue entry is leased by the referenced lease;
- no Review exists for the submission version;
- decision/finding validation passes;
- required prior finding resolutions are supplied for a revision.

Transaction effect:

```
create Review
create ReviewFinding records
create FindingResolution records
ReviewLease.status -> consumed
ReviewLease.close_reason -> review_recorded
ReviewQueueEntry.queue_state -> closed
ReviewQueueEntry.closed_reason -> review_recorded
ReviewQueueEntry.active_lease_id -> null
apply decision-specific Task and TaskAssignment changes
write all audit events
commit
```

No partial Review may survive if any decision-specific state change fails.

---

12. Decision Contracts

12.1 Accept

Validation:

- no unresolved prior blocking finding;
- any new findings are advisory only;
- decision reason optional.

Effect:

```
Review(decision=accept) created
queue entry closed
lease consumed
Task.status -> accepted
TaskAssignment.status -> completed
emit ReviewAccepted
emit ContributionRecordRequested
```

`ContributionRecordRequested` is an integration event. The review implementation MUST NOT create payment or reputation records.

12.2 Needs revision

Validation:

- decision reason required;
- at least one finding required;
- at least one finding is blocking.

Effect:

```
Review(decision=needs_revision) created
findings created
queue entry closed
lease consumed
Task.status -> needs_revision
TaskAssignment remains active
emit ReviewNeedsRevision
```

If the contributor continues:

```
Task.status -> in_progress
create SubmissionVersion(N+1)
create required SubmissionFindingResponse records
run existing finalization and checker spine
if allowed -> create preferred ReviewQueueEntry for prior reviewer
```

12.3 Reject

Validation:

- decision reason required.

Atomic effect:

```
Review(decision=reject) created
queue entry closed
lease consumed

TaskAssignment.status -> blocked
TaskAssignment.blocked_at -> now
TaskAssignment.blocked_reason_review_id -> Review.id

Task.status -> closed
Task.closed_reason -> review_rejected

emit ReviewRejected
emit TaskAssignmentBlocked
emit TaskClosed
```

Reject is terminal in v0.1:

- no new submission version may be created;
- the blocked submitter cannot reclaim the task;
- the contributor's project role grant remains unchanged;
- other project tasks remain unaffected.

Future `reopen_to_pool` behaviour is not implemented here.

13. Revision Routing

13.1 Same reviewer claims within preference window

```
R1 reviews v1 -> needs_revision
Submitter creates v2
v2 passes durable checker
queue entry v2 -> preferred to R1
R1 claims before preference expiry
R1 records Review(v2)
Review(v2).prior_review_id = Review(v1).id
```

13.2 Different reviewer after preference expiry

```
R1 reviews v1 -> needs_revision
Submitter creates v2
v2 enters R1 preferred backlog
R1 does not claim before preference expiry
entry becomes open with original queue age preserved
R2 claims
R2 receives complete prior chain
R2 records Review(v2)
Review(v2).prior_review_id = Review(v1).id
```

The prior chain is independent of reviewer identity.

13.3 Preferred reviewer becomes ineligible

Preference MUST be cleared immediately when Workstream detects that the preferred reviewer:

- lost or had their project reviewer grant revoked;
- became the submitter due to corrected attribution;
- was suspended;
- became otherwise disallowed by locked project policy.

Effect:

```
entry remains pending
routing_mode -> open
preferred fields -> null
available_since -> now
first_queued_at -> unchanged
record ReviewerPreferenceInvalidated
```

13.4 Preferred reviewer has another active lease

The entry remains in that reviewer's preferred backlog until:

- they complete/release their active lease and claim it;
- they decline preference;
- the preference window expires;
- an Admin clears the preference.

Preferred routing does not bypass the one-active-lease capacity limit.

---

14. Grant Revocation During an Active Lease

Grant revocation and lease recovery MUST be coordinated.

When a reviewer grant is revoked:

1. The grant becomes revoked immediately.

2. Any active lease held under that grant becomes `revoked`.
3. The queue entry returns to `pending/open`.
4. Preferred routing is cleared.
5. The reviewer can no longer record a canonical decision.
6. Historical lease and audit records remain.

The grant-revocation transaction SHOULD perform this recovery immediately. A background reconciliation worker MUST also detect and repair any missed active lease.

---

15. API Contract

Endpoint names may be adapted to existing router conventions, but capability boundaries and error semantics are locked.

15.1 Contributor-role grants

```
POST /v1/projects/{project_id}/role-grants
GET  /v1/projects/{project_id}/role-grants
POST /v1/projects/{project_id}/role-grants/{grant_id}/revoke
```

Create request:

```
{
  "contributor_id": "uuid",
  "role": "reviewer",
  "qualification_snapshot_ref": "uuid",
  "grant_method": "manual"
}
```

15.2 Reviewer queue

```
GET /v1/reviews/queue/mine
GET /v1/projects/{project_id}/reviews/queue
```

Reviewer response separates routing views:

```
{
  "active_lease": null,
  "preferred_backlog": [],
  "open_queue": []
}
```

15.3 Claim

```
POST /v1/reviews/queue/{queue_entry_id}/claim
```

Response:

```
{
  "queue_entry_id": "uuid",
  "lease_id": "uuid",
  "lease_status": "active",
  "claimed_at": "timestamp",
  "expires_at": "timestamp"
}
```

15.4 Release or decline

```
POST /v1/reviews/leases/{lease_id}/release
POST /v1/reviews/queue/{queue_entry_id}/decline-preference
```

15.5 Record decision

```
POST /v1/reviews/leases/{lease_id}/decision
```

Needs-revision request example:

```
{
  "decision": "needs_revision",
  "decision_reason": "Two blocking requirements remain unresolved.",
  "findings": [
    {
      "description": "Required verifier evidence is missing.",
      "severity": "blocking",
      "evidence_refs": ["cid-or-artifact-ref"]
    }
  ],
  "prior_finding_resolutions": []
}
```

Revision review example:

```
{
  "decision": "accept",
  "decision_reason": null,
  "findings": [],
  "prior_finding_resolutions": [
    {
      "finding_id": "uuid",
      "resolution_status": "resolved",
      "rationale": "Verifier evidence is present and valid."
    }
  ]
}
```

15.6 Read immutable chain

```
GET /v1/tasks/{task_id}/review-chain
GET /v1/submission-versions/{version_id}/review
```

The task chain response includes versions, reviews, findings, finding responses, resolutions, and relevant lease summaries in chronological order.

16. Error Contract

HTTP	Code	Condition
403	`unauthorized_actor`	Missing required Admin or Contributor authority
403	`reviewer_grant_required`	No active reviewer/both grant for project
403	`self_review_forbidden`	Reviewer is the submission creator
403	`lease_not_owned`	Decision/release actor does not own active lease
409	`reviewer_preference_active`	Another reviewer attempts claim during active preference window
409	`reviewer_capacity_reached`	Reviewer already holds an active lease
409	`queue_entry_not_pending`	Claim attempted on leased or closed entry
409	`queue_entry_not_leased`	Decision attempted when entry is not leased
409	`lease_expired`	Decision attempted after lease expiry
409	`review_already_recorded`	Review already exists for submission version
409	`task_assignment_blocked`	Submitter attempts to reclaim rejected task
409	`preference_not_owned`	Actor tries to decline someone else's preference
422	`decision_reason_required`	Missing reason for needs_revision or reject
422	`findings_required`	needs_revision contains no findings
422	`blocking_finding_required`	needs_revision has no blocking finding
422	`blocking_finding_on_accept`	accept includes a new blocking finding
422	`finding_resolution_required`	Prior blocking finding was not evaluated
422	`unresolved_finding_on_accept`	accept attempted with unresolved prior blocking finding

Errors MUST use the existing structured Workstream error envelope.

17. Database Constraints and Indexes

The implementation MUST enforce core invariants in the database, not only in service code.

Required constraints:

```
UNIQUE ReviewQueueEntry(submission_version_id)
UNIQUE Review(submission_version_id)
UNIQUE SubmissionVersion(task_id, version_number)
UNIQUE FindingResolution(finding_id, submission_version_id)
```

Required partial uniqueness:

```
UNIQUE ReviewLease(review_queue_entry_id) WHERE status = 'active'
UNIQUE ReviewLease(reviewer_id) WHERE status = 'active'
```

Required indexes:

```
ReviewQueueEntry(project_id, queue_state, routing_mode, first_queued_at)
ReviewQueueEntry(preferred_reviewer_id, queue_state, preference_expires_at)
ReviewLease(status, expires_at)
ProjectRoleGrant(project_id, contributor_id, status, role)
SubmissionVersion(task_id, version_number)
Review(prior_review_id)
ReviewFinding(review_id)
FindingResolution(finding_id, submission_version_id)
TaskAssignment(task_id, contributor_id, status)
```

Check constraints SHOULD enforce state-field compatibility for queue entries and leases.

18. Concurrency and Transaction Requirements

18.1 Claim race

If two reviewers claim the same entry concurrently, exactly one succeeds. The other receives `409 queue_entry_not_pending` or an equivalent conflict.

18.2 Capacity race

If one reviewer attempts to claim two entries concurrently, exactly one active lease may be created. The other receives `409 reviewer_capacity_reached`.

18.3 Decision versus expiry race

The server evaluates lease validity using one transaction and database time.

- If decision commit locks the active lease before the expiry transition, and the database time is not past `expires_at`, the decision may succeed.
- If expiry wins or the database time is past `expires_at`, decision fails with `lease_expired`.

18.4 Decision idempotency

The decision endpoint MUST require or derive an idempotency key. Repeating the same request returns the existing Review. Reusing the same key with a different payload returns an idempotency mismatch.

18.5 Reject atomicity

Review creation, queue closure, lease consumption, assignment blocking, task closure, and audit writes happen in one transaction.

18.6 Needs-revision atomicity

Review, findings, prior finding resolutions, queue closure, lease consumption, task transition, and audit writes happen in one transaction.

19. Background Processing and Recovery

19.1 Preference-expiry sweep

Periodically finds:

```
queue_state = pending
routing_mode = preferred
preference_expires_at < now
```

and opens the entries idempotently.

19.2 Lease-expiry sweep

Periodically finds:

```
ReviewLease.status = active
expires_at < now
no canonical Review exists
```

and returns each entry to the open pool idempotently.

19.3 Lazy recovery

Queue list, claim, and decision operations SHOULD also detect stale preference/lease state so correctness does not depend exclusively on the sweep schedule.

19.4 Orphan reconciliation

A reconciliation job detects:

- leased queue entry without an active lease;
- active lease whose queue entry does not reference it;
- consumed lease without a Review;
- closed `review_recorded` queue entry without a Review;
- revoked grant with an active lease;
- Review whose task/submission chain does not match.

It MUST alert operators and use explicit compensating events rather than silent history edits.

---

20. Audit Events

The following event types are required.

Authority

- `ProjectRoleGrantIssued`
- `ProjectRoleGrantRevoked`
- `ProjectRoleQualificationSnapshotCreated`

Routing

- `ReviewQueueEntryCreated`
- `ReviewRoutedToPreferredReviewer`
- `ReviewerPreferenceExpired`
- `ReviewerPreferenceInvalidated`
- `ReviewerDeclinedPreference`
- `ReviewQueueEntryOpened`
- `ReviewQueueEntryClosed`

Lease

- `ReviewerClaimedTask`
- `ReviewerReleasedTask`
- `ReviewerLeaseExpired`

- `ReviewerLeaseRevoked`
- `ReviewerLeaseConsumed`
- `ReviewerLeaseForceReleased`

Decision and revision

- `ReviewRecorded`
- `ReviewAccepted`
- `ReviewNeedsRevision`
- `ReviewRejected`
- `ReviewFindingCreated`
- `FindingResolutionRecorded`
- `SubmissionFindingResponseCreated`

Task and assignment

- `TaskAssignmentCompleted`
- `TaskAssignmentBlocked`
- `TaskClosed`
- `AdminOverrideApplied`

Every event records:

```
event_id: uuid
event_type: string
occurred_at: timestamp
actor_id: actor_id | system_actor_id
project_id: uuid
task_id: uuid | null
submission_version_id: uuid | null
review_queue_entry_id: uuid | null
review_lease_id: uuid | null
review_id: uuid | null
reason: string | null
policy_version: string
correlation_id: string
```

Lease-expiry and voluntary-release events remain separate because future reputation policy may treat them differently.

---

21. Notifications and Operational Views

Notification delivery is an adapter concern, but the lifecycle emits events for:

- preferred review routed to reviewer;
- preference nearing expiry;
- lease nearing expiry;
- lease expired;
- review needs revision;
- review accepted;
- review rejected;

- reviewer grant revoked while work is leased.

Reviewer view

Must show:

- current active lease and remaining time;
- preferred backlog count and oldest age;
- open eligible queue count;
- prior submission/review chain for claimed work;
- unresolved findings and submitter responses.

Admin view

Must show:

- open FIFO depth per project;
- preferred backlog per reviewer;
- oldest preferred entry age;
- active lease count;
- leases approaching expiry;
- expired and voluntarily released lease counts;
- entries reopened because reviewer preference became invalid;
- review turnaround distributions.

The system MUST NOT report the queue as empty when preferred reviewer backlogs still contain pending work.

22. Observability

Required metrics include:

```
review_queue_open_depth{project_id}
review_queue_preferred_depth{project_id}
review_queue_oldest_open_age_seconds{project_id}
review_preferred_oldest_age_seconds{project_id}
review_active_lease_count
review_lease_expired_total{project_id}
review_lease_released_total{project_id}
review_claim_conflict_total{reason}
review_decision_total{decision, project_id}
review_turnaround_seconds{project_id, decision}
review_revision_count{project_id}
review_preference_fallback_total{project_id}
```

Per-reviewer metrics should be available in authorized operational views but should not create uncontrolled high-cardinality public metric labels.

Structured logs MUST exclude tokens, credentials, raw private artifact content, and sensitive source paths.

23. Security Requirements

- Review authority is checked from current Workstream state, not token role claims.

- No self-review in v0.1.
- Admin privileges do not bypass reviewer qualification.
- Reviewer decision requires an owned, active, unexpired lease.
- Grant revocation invalidates decision authority immediately.
- Queue and chain reads are project-authorized.
- Reviewer can read only entries they may claim, entries preferred to them, their active lease, and permitted historical context.
- Admin override requires a reason and dedicated audit event.
- Immutable records are corrected only through new compensating records/events.
- All evidence references shown during review must use the task's locked guide, policy, checker, and artifact context.

24. End-to-End Reference Sequences

24.1 First submission accepted

```
1. Finalized submission creates SubmissionVersion(v1).
2. Durable checker completes with allow_review.
3. Open ReviewQueueEntry(v1) is created.
4. Eligible Reviewer R1 claims atomically.
5. ReviewLease L1 becomes active.
6. R1 records accept before L1 expires.
7. Review(v1) is created.
8. L1 becomes consumed.
9. Queue entry closes with review_recorded.
10. Task becomes accepted.
11. Submitter TaskAssignment becomes completed.
12. ContributionRecordRequested is emitted.
```

24.2 Needs revision and same reviewer returns

```
1. R1 reviews v1 as needs_revision with blocking findings.
2. Task becomes needs_revision.
3. Submitter creates v2 plus responses to every blocking finding.
4. v2 passes the durable checker.
5. ReviewQueueEntry(v2) is preferred to R1.
6. R1 sees v2 in their preferred backlog.
7. R1 claims within the preference window.
8. R1 records finding resolutions and a new decision.
9. Review(v2).prior_review_id points to Review(v1).
```

24.3 Needs revision and another reviewer takes over

```
1. v2 is preferred to R1.
2. R1 does not claim before preference expiry.
3. Preference clears; first_queued_at remains unchanged.
4. v2 becomes visible in the open eligible FIFO pool.
5. R2 claims.
6. R2 receives the complete v1/v2 submission and review chain.
7. R2 records the next Review and finding resolutions.
```

24.4 Claimed review expires

```
1. R1 claims queue entry Q1 and receives L1.
2. L1 expires without Review creation.
3. L1 becomes expired.
4. Q1 returns to pending/open.
5. Q1 preserves first_queued_at and updates available_since.
6. ReviewerLeaseExpired is recorded.
7. R1 has no renewed preference but may later claim from the open pool.
```

24.5 Reject

```
1. R1 holds active lease L1.
2. R1 records reject with a required reason.
3. Review is created and L1 consumed.
4. Queue entry closes.
5. Submitter TaskAssignment becomes blocked.
6. Task closes with review_rejected.
7. No new submission version may be created in v0.1.
```

---

25. Conformance Tests

25.1 Authority tests

- Submitter-only grant cannot claim review work.
- Reviewer-only grant cannot claim submission task work.
- `both` may perform both functions but cannot self-review.
- Reviewer grant on Project A cannot claim Project B review work.
- Admin without contributor reviewer grant cannot review.
- External reviewer behaves identically after local contributor onboarding and grant.
- Revoked grant cannot claim.
- Grant revoked during lease prevents decision.

25.2 Queue-routing tests

- First submission enters open pool.
- Revision enters prior reviewer's preferred backlog.
- Non-preferred reviewer receives `reviewer_preference_active` during the window.
- Preferred reviewer may claim during the window.
- Preference expiry opens the entry.
- Preference expiry preserves `first_queued_at`.
- Preferred reviewer decline opens the entry immediately.
- Admin assignment creates time-limited preferred routing.
- General queue may be empty while preferred backlog is non-empty.
- Reviewer next-work ordering chooses preferred backlog before open pool.

25.3 Lease tests

- Two concurrent reviewers cannot claim the same entry.
- One reviewer cannot hold two active leases.

- Reviewer release returns entry to pending/open.
- Lease expiry returns entry to pending/open.
- Lease expiry clears stickiness.
- Expired reviewer may later claim from open pool if eligible.
- Expired lease cannot record decision.
- Release and expiry emit distinct audit events.
- Decision and expiry race resolves deterministically using database time.

25.4 Review tests

- A second Review cannot be stored for the same submission version.
- Review references the correct queue entry and lease.
- `needs_revision` without findings fails.
- `needs_revision` without a blocking finding fails.
- `accept` with a new blocking finding fails.
- `reject` without a decision reason fails.
- Decision endpoint is idempotent.
- Same idempotency key with different payload fails.
- Review cannot be edited after creation.

25.5 Revision tests

- Version numbers increase monotonically.
- v2 points to v1.
- Review(v2) points to Review(v1) regardless of reviewer identity.
- Revised submission requires a response to each unresolved blocking finding.
- Original finding remains unchanged.
- Resolution is stored as a new FindingResolution.
- Accept fails if a prior blocking finding remains unresolved.
- Complete history is traversable from latest version backward.

25.6 Reject tests

- Reject closes the queue entry.
- Reject consumes the lease.
- Reject blocks the submitter's TaskAssignment.
- Blocked submitter cannot reclaim the task.
- ProjectRoleGrant remains active.
- Other task assignments remain unaffected.
- Task closes with `review_rejected`.
- No later submission version may be created.

25.7 Recovery tests

- Preference sweep is idempotent.
- Lease sweep is idempotent.
- Lazy access repairs stale expired preference/lease state.
- Reconciliation detects leased entry without active lease.
- Reconciliation detects consumed lease without Review.
- Reconciliation never silently deletes or rewrites immutable history.

26. Implementation Delivery Order

Phase 1: Schema and invariants

- Project role qualification snapshot
- ProjectRoleGrant updates
- SubmissionVersion constraints
- SubmissionFindingResponse
- ReviewQueueEntry
- ReviewLease
- Review
- ReviewFinding
- FindingResolution
- TaskAssignment extension
- Required unique and partial indexes

Exit condition: migrations and model-level constraint tests pass.

Phase 2: Authority and queue creation

- Admin grant/revoke endpoints
- reviewer eligibility service
- checker-admission to queue-entry creation
- preferred/open routing fields
- reviewer/admin queue reads

Exit condition: authority and routing conformance tests pass.

Phase 3: Claims and timers

- atomic claim
- global one-active-lease enforcement
- reviewer release
- preferred decline
- preference-expiry sweep
- lease-expiry sweep

- lazy recovery

Exit condition: claim concurrency and timer tests pass.

Phase 4: Immutable decisions

- decision validation
- immutable Review and ReviewFinding writes
- FindingResolution writes
- accept and needs-revision transitions
- decision idempotency

Exit condition: review and revision tests pass.

Phase 5: Reject and operational hardening

- assignment blocking
- terminal task closure
- grant-revocation lease recovery
- Admin force-release
- reconciliation worker
- metrics, dashboards, and notifications

Exit condition: reject, recovery, security, and observability tests pass.

Phase 6: Live API drill

Run one HTTP-visible reference flow covering:

```
first submission
-> preferred/open routing
-> claim
-> needs_revision
-> revised submission
-> preferred reviewer expiry or takeover
-> accept
```

Run a separate terminal reject flow and one lease-expiry recovery flow.

Exit condition: privacy-safe evidence report and internal trust bundle are approved.

---

27. Definition of Done

The review lifecycle is complete only when:

- reviewer authority is based on explicit current project grants;
- self-review is impossible in v0.1;
- one reviewer cannot hold multiple active leases;
- preferred reviewer backlogs and the open queue are separately visible;
- first submissions route open and revisions route back to the prior reviewer;
- preference and lease timers operate independently;
- FIFO age is preserved when reviewer delay causes requeue;

- every lease attempt remains auditable;
- Review, ReviewFinding, FindingResolution, and SubmissionVersion are immutable;
- revision replays every unresolved blocking finding;
- accept, needs_revision, and reject transitions are atomic;
- reject blocks only the submitter-task assignment and closes the task in v0.1;
- runtime state and the HTTP-visible audit trail agree;
- conformance tests and live drills prove normal, failure, timeout, and concurrency paths.

---

28. Explicitly Out of Scope

- Automated reviewer-grant issuance from reputation thresholds.
- Reputation scoring formulas.
- Skills/reputation-based automated queue ranking.
- Multiple simultaneous review leases per reviewer.
- Self-review policy.
- Reviewer bidding or marketplace allocation.
- ContributionRecord implementation.
- Payment-status implementation.
- ReputationEvent implementation.
- Reject reassignment through `reopen_to_pool`.
- Public or cross-organization reviewer federation.
- Changes to Identity Issuer internal architecture.
- Changes to Flow Node artifact-storage internals.

---

29. Future Additive Directions

The following may be added without changing the v0.1 authority and immutable-chain model:

- automated `ProjectRoleGrant` issuance from a versioned policy engine;
- reviewer capacity greater than one under project policy;
- routing scores using skill, reputation, turnaround, and conflict data;
- separate submitter and reviewer reputation dimensions;
- project-level reject policy `reopen_to_pool`;
- reviewer cooldown after lease expiry;
- review escalation and adjudication;
- cross-organization expert-reviewer onboarding;
- evaluation-sprint batching if Workstream later needs sprint-level ownership and reporting.

None of these future directions may silently rewrite existing grants, queue events, leases, submissions, reviews, findings, or resolutions.

---

30. Coding-Agent Handoff Rules

The coding agent MUST:

1. Treat this specification as the review-lifecycle authority.
2. Inspect existing Workstream status enums, models, services, repositories, routers, audit patterns, and error envelopes before editing.
3. Preserve the proven intake spine and extend it only after `review_pending`.
4. Reuse existing actor-resolution and authorization patterns.
5. Implement database constraints before relying on service checks.
6. Keep routers thin; lifecycle rules belong in services/domain guards.
7. Use database transactions for claims and decisions.
8. Use the existing async worker boundary for sweeps and reconciliation.
9. Add focused unit, service, API, concurrency, authorization, and failure-mode tests.
10. Keep contribution, payment, and reputation implementation outside this change.
11. Produce evidence showing exactly what changed, what passed, and what remains unimplemented.
12. Stop and request architecture confirmation if existing code requires changing a locked decision in this document.